
Day 30 – Flyweight Design Pattern

Introduction

👉 Flyweight Design Pattern is used to reduce memory usage when we need to create a very large number of similar objects.

- Flyweight pattern that is used to minimize memory usage by sharing common parts of state between multiple objects instead of storing all data in each object.

Let's Take an Example:

- Suppose hum game bana rahe hain
- Bahot saare objects create ho rahe hain
- RAM limited hoti hai
- Agar har object ka pura data alag store karenge toh memory issue aa sakta hai

Game Example

- You are developing a game. You have spaceship which shoots laser and many asteroids are also coming and you have to shoot that asteroids.

Scenario

Spaceship laser shoot karta hai.

Bahot saare asteroids aa rahe hain.

Difficulty level ke according:

Easy → 3-4 asteroids
Hard → hundreds of asteroids

👉 Agar memory optimize nahi hui toh:

Game crash ho sakta hai

Problem Without Flyweight

Suppose game me:

1000 Asteroids

Aur har asteroid me same properties store ho:

```
length  
width  
weight  
color  
texture
```

👉 Toh unnecessary memory waste hogi.

Solution

👉 Reuse already created objects instead of creating new objects again and again.

Instead of:

Create 1000 same objects

↓

Reuse common objects

Why Flyweight Needed?

👉 Common properties ko share karo.

👉 Sirf unique properties alag rakho.

- Object ko 2 parts me divide karo:

1. Intrinsic Properties
2. Extrinsic Properties

Intrinsic Properties

👉 Shared properties

👉 Reusable properties

```
length  
width  
weight
```

```
color  
texture
```

Ye multiple asteroids me same ho sakti hain.

Extrinsic Properties

👉 Unique properties

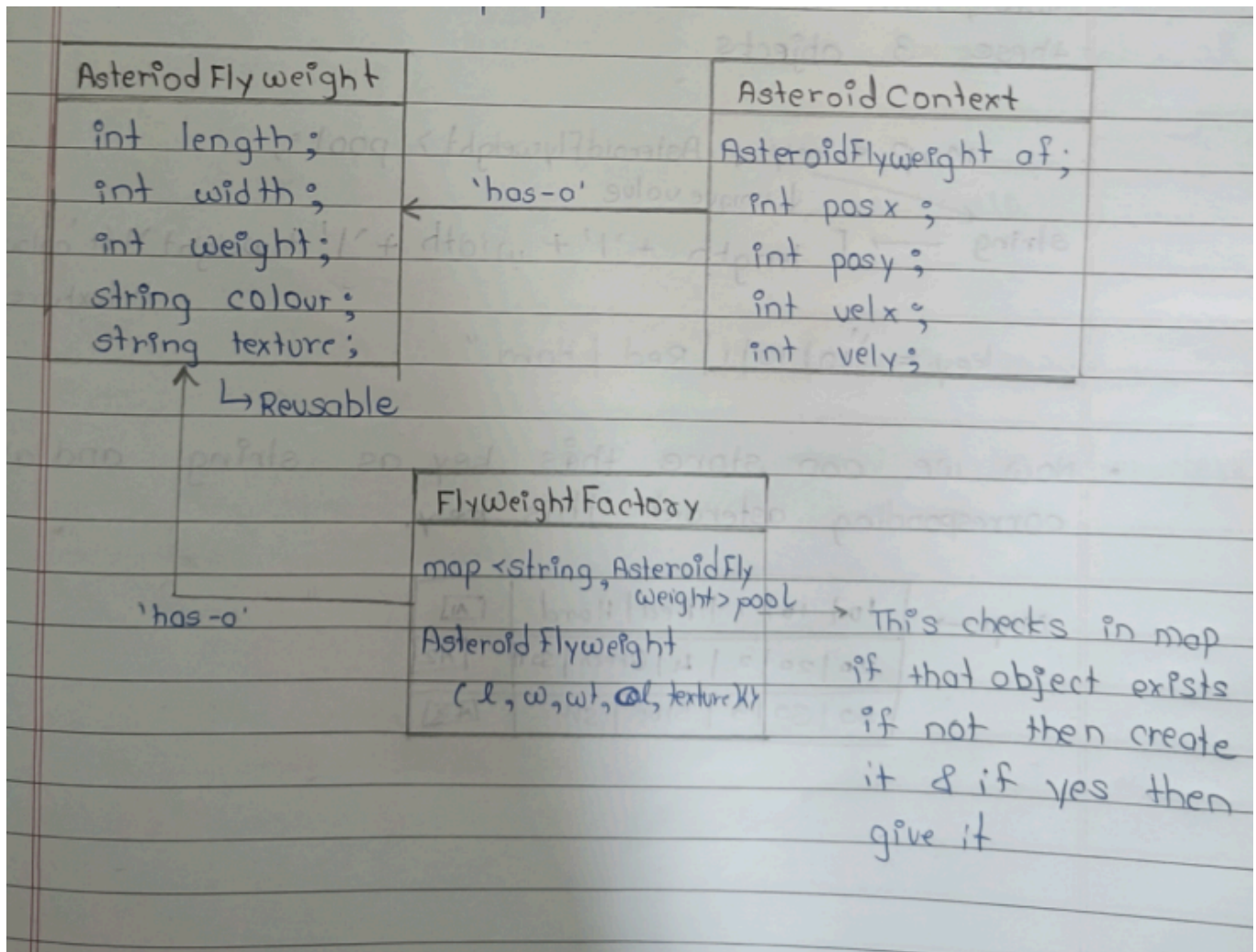
👉 Har object me alag

```
position (x,y)  
velocity (vx, vy)
```

Asteroid Example Before Optimization

properties	Asteroid
All have limited values	int length; int width; int weight; string colour; string texture;
asteroid position on screen	int pos x, pos y;
velocity of asteroid	int vel x, vely;

After Flyweight Optimization



Understanding the Diagram

AsteroidFlyweight

Shared reusable object

Contains:

```
length
width
weight
color
texture
```

Same object multiple asteroids use kareng.

AsteroidContext

Runtime unique object

Contains:

```
position  
velocity
```

■ Flyweight Factory

Factory ka kaam:

```
Check if object already exists
```

If exists:

```
Return existing object
```

Else:

```
Create new object  
Store in pool  
Return object
```

■ Internal Working

Factory ek map maintain karta hai.

```
map<key, AsteroidFlyweight> pool
```

■ How Key is Generated?

```
key =  
length + width + weight + color + texture
```

Example:

```
10|10|1|Red|Hard
```

■ Example Pool

Map:

```
10|10|1|Red|Hard → A1  
20|20|2|Green|Soft → A2  
30|30|3|Blue|SH → A3
```

Huge Memory Optimization

Suppose combinations:

```
length → 10,20,30  
width → 10,20,30  
weight → 1,2,3  
color → R,G,B  
texture → H,S,SH
```

👉 Total combinations become limited.

Instead of:

1000 objects

We may create only:

3 or few flyweight objects

⚠ Flyweight Objects Should be Immutable

Because:

If shared object changes,
all users get affected

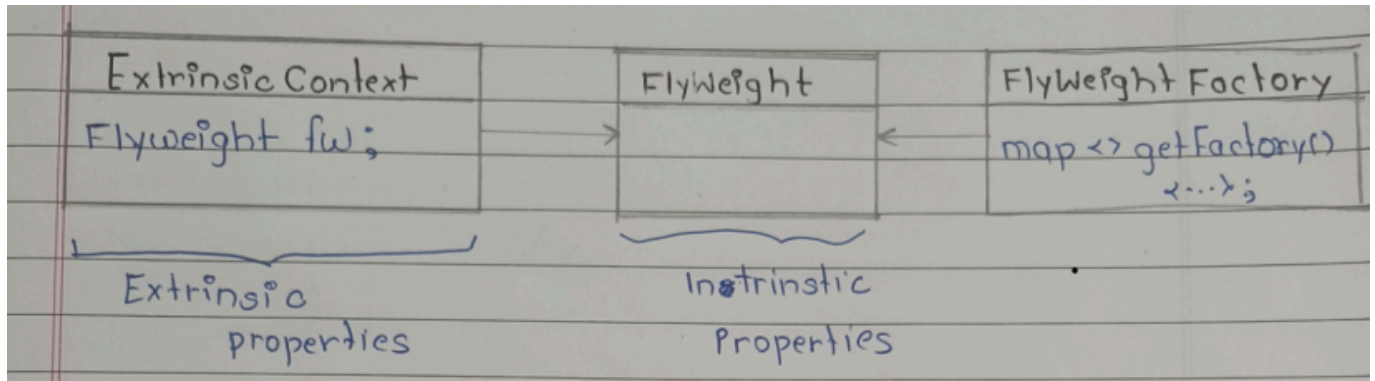
Example:

A1 used by 500 asteroids

If color changed:

All 500 asteroids change

Standard UML



UML Explanation

Flyweight

Stores:

Intrinsic properties

ExtrinsicContext

Stores:

Extrinsic properties

FlyweightFactory

Responsible for:

Creating / reusing objects

Standard Definition



Flyweight uses sharing to support large numbers of fine-grained objects efficiently.

Real World Examples

Games (GTA-5)

Used to optimize RAM usage.

Text Editors

Characters share same:

font
size
style

But different:

position

Real Life Analogy

Suppose school me:

1000 students

Uniform same hai:

shirt
pant
tie

👉 Uniform shared ho sakta hai.

But unique things:


```
name  
roll no  
seat no
```

alag honge.



Advantages

- ✓ Huge memory optimization
 - ✓ Reuse objects
 - ✓ Faster performance
 - ✓ Efficient large-scale systems
 - ✓ Less RAM usage
-



Disadvantages

- ✗ Code complexity increase
 - ✗ Factory management needed
 - ✗ Hard debugging
 - ✗ Immutable objects required
-



When to Use?

Use Flyweight when:

- ✓ Bahot saare similar objects ho
 - ✓ Memory optimization needed ho
 - ✓ Shared data possible ho
-



Interview One-Line



Flyweight pattern minimizes memory usage by sharing common object data.